ADVANCED URL SHORTENER WEB APPLICATION

Project Report

1. Introduction

With the rapid growth of internet usage, sharing long URLs has become a common challenge. Long web links are difficult to remember, copy, and share across platforms such as social media, emails, and messaging applications. A URL Shortener solves this problem by converting long web addresses into short and easy-to-share links.

This project focuses on developing an **Advanced URL Shortener Web Application** that not only shortens URLs but also provides user authentication functionality. Users can register, log in, generate shortened URLs, and view their personal URL history. The application is built using Flask for backend processing, SQLAlchemy ORM for database interaction, and HTML, CSS, and Bootstrap for frontend design.

2. Project Objectives

The main objectives of this project are:

- To develop a secure login and signup system for users.
- To allow authenticated users to shorten URLs.
- To store original and shortened URLs in a database.
- To provide a copy button for easy sharing of shortened URLs.
- To maintain separate URL histories for each user.
- To design a clean and centered user interface for better user experience.
- To implement URL validation to avoid invalid inputs.

3. Technologies Used

Frontend Technologies

- **HTML**: Used for creating the structure of web pages.
- **CSS**: Used for styling the interface and centering all content on the page.
- **Bootstrap**: Used for responsive design and better UI components.

Backend Technologies

- **Python**: Used as the main programming language.
- **Flask**: Lightweight web framework used to handle routing, requests, and responses.

Database and ORM

- **SQLite**: Lightweight relational database used to store user and URL information.
- **SQLAlchemy ORM**: Used to interact with the database using Python objects instead of SQL queries.

Authentication Library

- **Flask-Login**: Used to manage user login sessions and authentication.

4. System Architecture

The application follows a client-server architecture:

- The frontend provides user interfaces for login, signup, and URL shortening.
- The Flask backend processes requests, validates inputs, and handles business logic.
- SQLAlchemy ORM manages database interactions.
- Flask-Login handles user session management.

This architecture ensures modularity, security, and maintainability.

5. Application Workflow

The complete workflow of the application is as follows:

1. The user opens the home page of the application.
2. The user selects either login or signup.
3. During signup, the system checks whether the username already exists and validates the username length.
4. After successful signup, the user is redirected to the login page.
5. The user logs in using valid credentials.
6. After login, the user is redirected to the dashboard page.
7. The user enters a long URL and clicks the “Shorten URL” button.
8. The backend validates the URL format.
9. A random short code is generated for the URL.
10. The shortened URL is stored in the database along with the user ID.
11. The shortened URL is displayed on the dashboard.
12. The user can copy the shortened URL using the copy button.
13. When the shortened URL is accessed, the system redirects to the original URL.
14. The dashboard also displays the history of all URLs shortened by the logged-in user.

6. User Authentication System

The application includes a login and signup system using Flask-Login.

Signup Validation

The following validations are implemented:

- Username must be unique.
- Username length must be between 5 and 9 characters.

If the username already exists or does not meet length requirements, an error message is displayed.

Login Authentication

During login, the entered credentials are verified against the database. If valid, the user is logged in and redirected to the dashboard. Otherwise, an error message is shown.

This authentication system ensures that each user has a personalized and secure workspace.

7. URL Validation

URL validation is implemented to prevent invalid links from being processed. If the entered URL is not valid, the system displays an error message and stops further processing. This improves data accuracy and application reliability.

8. Database Design

The application uses two main tables:

User Table

This table stores user account information:

- User ID
- Username
- Password

URL Table

This table stores URL information:

- URL ID
- Original URL

- Short URL
- User ID (foreign key reference)

This relational structure ensures that each URL is associated with the user who created it.

9. Short URL Generation

Short URLs are generated using random combinations of alphanumeric characters. This approach ensures that the generated links are unique and difficult to predict. The generated short code is appended to the application's base URL to form the final shortened link.

10. Copy Button Functionality

A JavaScript-based copy button is implemented on the dashboard page. This feature allows users to copy the shortened URL with a single click using the browser clipboard API. This improves usability and saves time for users.

11. User Interface Design

The user interface is designed with simplicity and usability in mind. All content is centered on the screen to improve visual balance and readability. Input forms, buttons, and links are styled using CSS and Bootstrap to create a professional and modern look.

Separate pages are designed for:

- Home
- Signup
- Login
- Dashboard

Each page provides a clean and consistent layout.

12. Challenges Faced

During development, the following challenges were encountered:

- Implementing session-based authentication using Flask-Login.
- Managing database relationships between users and URLs.
- Validating user input correctly.
- Handling redirection logic for shortened URLs.
- Designing a responsive and centered user interface.

These challenges were resolved through careful debugging and structured implementation.

13. Testing and Results

The application was tested using multiple scenarios:

- Valid and invalid URL inputs
- Duplicate username registration attempts
- Correct and incorrect login credentials
- Multiple URL generations by different users
- Redirection using shortened URLs

All core features worked successfully and met the project requirements.